arXiv:2402.13598v2 [cs.CL] 9 Sep 2024

USER-LLM: Efficient LLM Contextualization with User Embeddings

Lin Ning[1*], Luyang Liu[1*†], Jiaxing Wu[1], Neo Wu[1], Devora Berlowitz[2], Sushant Prakash[2], Bradley Green[1], Shawn O'Banion[1], Jun Xie[1]

[1]Google DeepMind, [2]Google

Abstract

Large language models (LLMs) have achieved remarkable success across various domains, but effectively incorporating complex and potentially noisy user timeline data into LLMs remains a challenge. Current approaches often involve translating user timelines into text descriptions before feeding them to LLMs, which can be inefficient and may not fully capture the nuances of user behavior. Inspired by how LLMs are effectively integrated with images through direct embeddings, we propose USER-LLM, a novel framework that leverages user embeddings to directly contextualize LLMs with user history interactions. These embeddings, generated by a user encoder pretrained using self-supervised learning on diverse user interactions, capture latent user behaviors and interests as well as their evolution over time. We integrate these user embeddings with LLMs through cross-attention, enabling LLMs to dynamically adapt their responses based on the context of a user's past actions and preferences.

Our approach achieves significant efficiency gains by representing user timelines directly as embeddings, leading to substantial inference speedups of up to 78.1X. Comprehensive experiments on MovieLens, Amazon Review, and Google Local Review datasets demonstrate that USER-LLM outperforms text-prompt-based contextualization on tasks requiring deep user understanding, with improvements of up to 16.33%, particularly excelling on long sequences that capture subtle shifts in user behavior. Furthermore, the incorporation of Perceiver layers streamlines the integration between user encoders and LLMs, yielding additional computational savings.

1 Introduction

Large language models (LLMs) have revolutionized natural language processing (NLP) (Brown et al. 2020; Chowdhery et al. 2023; OpenAI 2023; Touvron et al. 2023; Anil et al. 2023; Google 2023) and demonstrated remarkable success across various domains, including language generation, summarization (Liu et al. 2023b; Basyal and Sanghvi 2023), and question answering (Kojima et al. 2022; Wei et al. 2023). This success has sparked significant interest in their potential to power the next generation of personalized AI agent - systems that understand and respond to individual needs and preferences. While LLMs have shown remarkable capabilities, their effectiveness often hinges on the quality and relevance of the data they are trained on. A key challenge lies in how to effectively incorporate the wealth of information contained within complex and potentially noisy user timeline data into these models. These timelines, spanning a wide range of interactions from textual input and search queries to media consumption, social media activity, location visits, etc., represent a rich source of user behavioral data. These data hold valuable insights crucial for deep user understanding and tailored user experience.

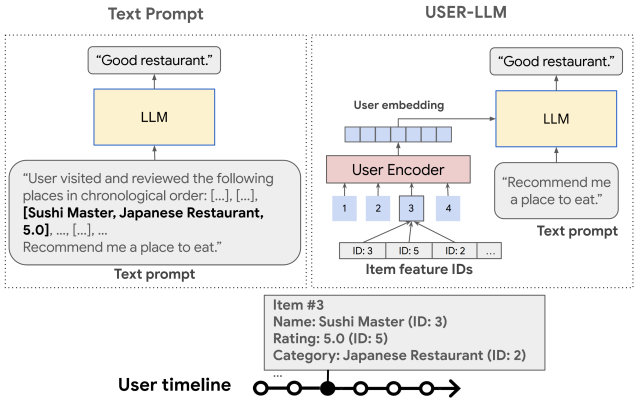

Figure 1: Illustration of two mechanisms to incorporate user interaction history data to LLMs. (1) Text Prompt: User history data are expressed in natural language and fed to LLMs via text prompt; (2) USER-LLM: User embeddings, distilled from user history data, are used to contextualize LLMs.

Existing approaches often translate user timelines into lengthy textual descriptions before feeding them into LLMs. This approach, while conceptually straightforward, leads to significantly computational overhead during inference due to the long context windows required to process these textual representation. Moreover, it may fail to capture the subtle nuances of user behavior and preferences essential for achieving truly personalized AI experiences.

Inspired by the effective integration of LLMs with other modalities, such as images (Alayrac et al. 2022; Chen et al. 2022a), by directly incorporating embeddings from modality-specific encoders, we propose a paradigm shift for incorporating user history into LLMs. Instead of relying on computationally expensive textual representations, we treat user

*Equal contributions.

†Correspondence to Luyang Liu {*luyangliu@google.com*}

timelines as a distinct modality, directly embedding them for LLM contextualization.

This embedding-based approach offers several key advantages. First and most, it significantly reduces the context window length required for LLM processing, leading to substantial improvements in inference efficiency. Second, it allows LLMs to directly capture the latent patterns and temporal dynamics within user interactions, potentially fostering a deeper understanding of individual users and their evolving needs.

To realize this vision, we introduce USER-LLM, a novel framework (1) that leverages user embeddings to directly contextualize LLMs. These embeddings, generated by a user encoder pretrained using self-supervised learning on diverse user interactions, encapsulate latent user behaviors, interests, and their evolution over time. We integrate these user embeddings with LLMs through cross-attention, enabling the models to dynamically adapt their responses based on the rich context of a user's past actions and preferences.

By representing user timelines directly as embeddings, we drastically reduce the computational burden on the LLM during inference, leading to inference speedups of up to 78.1X compared to text-prompt-based methods. Our comprehensive experiments across three datasets further demonstrate USER-LLM's ability to outperform text-prompt-based contextualization on tasks requiring deep user understanding, particularly with long context inputs.

Our **contributions** are four-fold:

- We introduce USER-LLM, a framework that leverages user embeddings to directly contextualize LLMs, enabling them to dynamically adapt to a user's past actions and preferences. USER-LLM supports various user encoder architectures and encoder-LLM co-training strategies.
- We conduct extensive experiments on three public datasets, demonstrating that USER-LLM significantly outperforms text-prompt-based contextualization methods, achieving up to 78.1X inference speedup and up to 16.33% improvement in performance on tasks requiring deep user understanding, particularly with long context inputs.
- We provide a comprehensive analysis of the impact of different user embedding generation architectures, encoder-LLM co-training strategies, and different ways to integrate embedding with LLMs on LLM personalization, offering valuable insights for future research.
- We delve into *cross-attention* mechanism for integrating user embeddings, exploring gated *cross-attention* to understand how user embeddings influence LLM behavior, and incorporate Perceiver layers for further efficiency gains.

2 Related Work

2.1 Multimodal LLM

Early work in multimodal LLMs primarily focuses on aligning images and text representations (e.g., CLIP (Radford et al. 2021), ImageBind (Girdhar et al. 2023)). Subsequently, fusion techniques leveraging *cross-attention* (e.g., Flamingo (Alayrac et al. 2022), Coca (Yu et al. 2022)) and *soft-prompt* (e.g., PaLI (Chen et al. 2022a), Palm-E (Driess et al. 2023)) emerged. Recent works, such as NextGPT (Wu et al. 2023b), OneLLM (Han et al. 2023), and Anymal (Moon et al. 2023), explored unified frameworks for diverse input modalities. Additionally, end-to-end training of multimodal models has gained attention, as seen in Gato (Reed et al. 2022) and Gemini (Gemini Team Google 2023). Inspired by these advances, USER-LLM focuses on directly contextualizing LLM with embeddings for user understanding.

2.2 Language Model based Personalization

Recent research has extensively explored incorporating user interaction history as text prompts for LLM personalization and recommendation (Petrov and Macdonald 2023; Kang et al. 2023; Xu, Hua, and Zhang 2023; Liu et al. 2023a; Lyu et al. 2023; Ji et al. 2023; Li et al. 2023; Wu et al. 2023a). However, directly feeding the entire user history can be computationally expensive due to the resulting long context lengths. Our approach leverages user embeddings for efficient LLM contextualization, significantly reducing computational overhead. While a recent study (Doddapaneni et al. 2024) also explored LLM personalization with user embeddings, it focused on deriving embeddings from text-based user activity and integrating them via *soft-prompt*. In contrast, USER-LLM utilizes efficient user activity tokens and sophisticated fusion techniques (e.g., *cross-attention*). We conduct a comprehensive evaluation across various datasets and tasks, demonstrating the effectiveness of USER-LLM.

2.3 Long Context in LLMs

Long context is crucial for models to incorporate long-term user data. Existing approaches for LLMs include extending the context window (e.g., via positional encodings remapping (Chen et al. 2023a; Jin et al. 2024) or by exposing relevant additional contexts available to a subset of attention layers (Tworkowski et al. 2023)), distill information from the long context (e.g., by using a separate retrieval model (Xu et al. 2023), a memory bank (Wang et al. 2023), or training special tokens (Ge et al. 2023; Mu, Li, and Goodman 2023; Chevalier et al. 2023)), modified attention computation (Ding et al. 2023; Liu, Zaharia, and Abbeel 2023; Chen et al. 2023b), and linear attention frameworks (e.g., structured state space models (Gu et al. 2023; Gu and Dao 2023; Tay et al. 2022)). Our method tackles long context by using a single token to represent each user event, which is compatible with existing long context techniques.

2.4 User Modeling

Traditional approaches for user modeling and personalization have widely employed techniques like dual encoders, self-supervised learning (Yao et al. 2021; Xie et al. 2022; Chen et al. 2022b; Xia et al. 2023; Cai et al. 2023; Yang et al. 2023), and pretrained models (e.g., BERT (Devlin et al. 2018), Bert4Rec (Sun et al. 2019), U-Bert (Qiu et al. 2021))) to generate effective user representations (Covington, Adams, and Sargin 2016a,b; Sountsov and Sarawagi 2016; Volkovs, Yu, and Poutanen 2017; Chidambaram et al. 2018; Gillick, Presta, and Tomar 2018; Yanga et al. 2018; Ma et al. 2018; Yi et al. 2019; Gillick et al. 2019; Yang et al. 2020; Jiang et al. 2020; Ning et al. 2021). While these approaches provide valuable insights into user behavior, they primarily operate outside the

realm of LLMs. In contrast, our work focuses specifically on enhancing LLM capabilities for personalization by directly contextualizing them with user representations. This allows us to explore the potential of LLMs as the foundation for powerful, adaptive AI agents. Therefore, direct comparisons with non-LLM based user modeling techniques are not within the scope of this work.

3 Methodology

This section introduces the USER-LLM framework for contextualizing LLM with user embeddings (Fig.2).

3.1 Problem Statement

Given a set of users $\mathcal{U}$ and a set of items $\mathcal{V}$, each user $u \in \mathcal{U}$ has a timeline sequence $S_u = (v_1^{(u)}, v_2^{(u)}, ..., v_{n_u}^{(u)})$ where each $v_i^{(u)} \in \mathcal{V}$ represents an item the user interacted with in chronological order. Each item v is associated with a set of features $\mathcal{M}$. Each feature $m \in \mathcal{M}$ has a vocabulary V_m that maps its values to integer IDs. For example, if items are movies, then the feature could be $\mathcal{M} = (name,\ genre,\ rating)$, where V_{name}, V_{genre} map text values to IDs and V_{rating} maps numerical ratings to IDs. Our goal is to develop a framework to contextualize a decode-only LLM with S_u, enabling it to generate personalized response A_u for user u given a query Q.

3.2 USER-LLM

As shown in Fig.2 (b), USER-LLM contains an ID-based user encoder and a decoder-only LLM. The **inference** includes two-stages: (1) the user encoder generates user embeddings; (2) the embeddings are integrated into the LLM through *cross-attention*, providing the LLM additional context and guidance to generate personalized responses.

User Embedding Generation USER-LLM leverages a Transformer-based encoder to generate user embeddings from ID-based interaction sequences.

Let us consider a scenario where each item has M features ($|\mathcal{M}| = M$). Representing each item with feature IDs, an ID-based user timeline sequence contains L items can be denoted as

$$s_u = [(x_{1,m_1}, ..., x_{1,m_M}), ..., (x_{L,m_1}, x_{L,m_M})]$$

where $x_{i,m_j} = V_{m_j}[m_j^{v_i}]$ denotes the ID of feature m_j for item v_i.

Each element ID has its unique embedding representation. To obtain a unified representation for an item, we combine embeddings from different features into a single embedding. Let $E_{m_j} \in \mathbb{R}^{|V_{m_j}| \times d_j}$ be the embedding matrix associated with V_{m_j}, where d_j is the embedding dimension for m_j. For item v_i, the fused embedding is $f_i = [E_{m_1}(x_{i,m_1}); E_{m_2}(x_{i,m_2}); ..., E_{m_M}(x_{i,m_M})]$, where $[\cdot;\cdot]$ denotes concatenation. To reduce the dimension of the fused embedding, we project f_i to f_i' with a dimension of d.

Autoregressive User Encoder As illustrated in Fig. 2 (a), an Autoregressive Transformer, denoted as $\mathcal{T}$, takes the fused embeddings $F' = [f_1', f_2', ..., f_L']$ into the Transformer decoder, and outputs embeddings $E_{s_u} = \mathcal{T}(F')$, where $E_{s_u} \in \mathbb{R}^{L \times d}$. These embeddings are used for LLM contextualization in the next stage.

Our framework is adaptable, supporting various user encoders. We've experimented with alternatives like Dual Encoders and achieved competitive results.

LLM Contextualization with User Embedding USER-LLM integrates user embeddings with LLMs using *cross-attention* (see Fig. 2 (b)). The output embeddings from the user encoder are down sampled by a projection layer, and then are cross-attended with intermediate text representations within the LLM, similar to how Flamingo (Alayrac et al. 2022) works.

Let E_{su} denote the encoder output, and $CrossAttn(Q, K, V)$ is the *cross-attention* mechanism. Note that for *cross-attention*, Q is LLM input, whereas K and V are compatible with the dimension of cross-input E_{su}. Let O_i and I_{i+1} denote the output of the i-th and the input of $(i+1)$-th self-attention layers, respectively. Our framework enables *cross-attention* as follows:

$$O_i = \text{LayerNorm}(O_i) \tag{1}$$

$$O_i = \text{CrossAttn}(O_i, E_{s_u}, E_{s_u}) + O_i \tag{2}$$

$$I_{i+1} = \text{FeedForward}(O_i) \tag{3}$$

In order to further investigate the *cross-attention* mechanisms between encoders and LLMs, we also implemented gated *cross-attention* in USER-LLM by inserting gated *cross-attention*-only layers between LLM's self-attention stack. Flamingo-style gating mechanism (Alayrac et al. 2022) was employed to control the contribution of *cross-attention* signal to LLM. That is, a scalar between 0 and 1 is introduced to equation (2):

$$O_i = \mathbf{tanh}(\boldsymbol{\alpha_i}) \times \text{CrossAttn}(O_i, E_{s_u}, E_{s_u}) + O_i \tag{4}$$

where α_i is a trainable, zero-initialized scalar gating the i-th *cross-attention* layer. Using this approach, we enabled *cross-attention* between encoder output and transformer layers in the LLM with layer-specific *cross-attention* weights regulated by α.

Note that our framework is adaptable to different integration mechanisms including prepending embeddings as *soft-prompt* (Lester, Al-Rfou, and Constant 2021).

3.3 Training Framework

To effectively extract information from user interaction data and teach LLM to understand the information encoded in the user embeddings, we propose a two-stage training framework: user-encoder pretraining and encoder-LLM finetuning.

User-encoder Pretraining In the first stage, we pretrain the Autoregressive Transformer on user interaction sequence so that it learns how to extract information from the input data. As described in Section 3.2, the Autoregressive Transformer takes in a sequence of ID-based user interactions and outputs embeddings E_{s_u}. For pretraining, the decoder output is projected back to the original feature dimensions using feature-specific projection matrices $W_{m_1} \in \mathbb{R}^{d \times d_1}, W_{m_2} \in \mathbb{R}^{d \times d_2}, ..., W_{m_i} \in \mathbb{R}^{d \times d_i}$: $h_{m_j,i} = W_{m_j} e_i$, where $e_i \in E$ is the i-th embedding output from the Transformer.

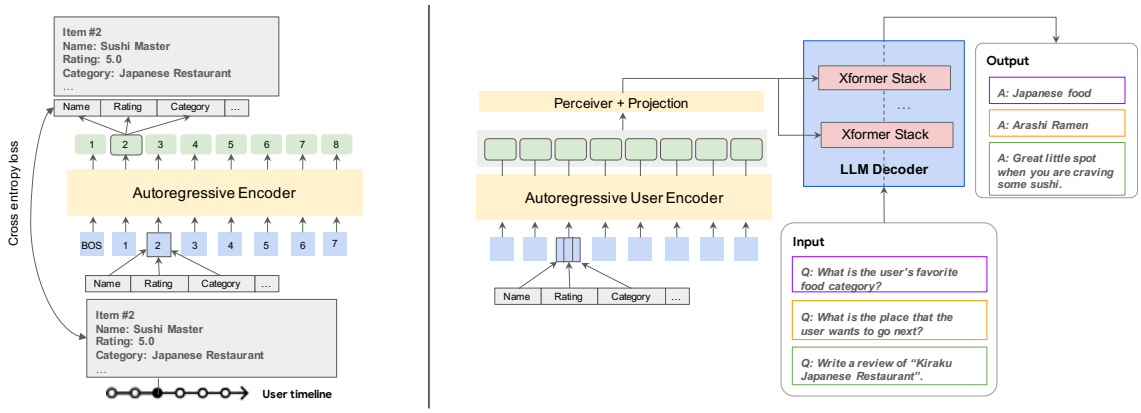

Figure 2: **Overview of USER-LLM**. (a) Autoregressive Transformer encoder pretraining. (b)LLM contextualization with user embeddings. Features from a user's interaction history are encoded by the Autoregressive User Encoder and then integrated into the language model via *cross-attention*.

Loss Calculation The cross-entropy loss for each feature is calculated using the logits of the feature-specific projection:

$$\mathcal{L}_{m_j} = -\frac{1}{L}\sum_{i=1}^{L}\log(\text{softmax}(h_{m_j,i}))(f'_{m_j,i+1})$$

where $f'_{m_j,i+1}$ is the projected fused embedding for feature m_j at step $i+1$. The overall loss is the average of cross-entropy losses across features: $\mathcal{L} = \frac{1}{i}\sum_{j=1}^{i}\mathcal{L}_{m_j}$

Encoder-LLM Finetuning In the second stage, we connected pretrained Autoregressive Transformer and a decode-only LLM with *cross-attention* and finetune the entire USER-LLM model to align user embeddings and LLM's text representations. The model input has two parts: an ID-based user interaction sequence as the input to the Autoregressive Transformer ($X_{\mathcal{T}} = s_u$) and a query as the input to the LLM ($X_{LLM} = Q$).

USER-LLM offers flexible training strategies for Encoder-LLM finetuning to tailor its performance for diverse use cases. To thoroughly explore USER-LLM's capabilities, we investigated four strategies targeting different model components:

- **Full**: Finetuning the entire model (LLM, user encoder, and projection layers) enables maximum parameter adaptation to user interactions, revealing the upper bounds of USER-LLM's personalization potential.
- **Enc**: Finetuning the user encoder and projection layers (LLM frozen) offers an efficient LLM contextualization approach while preventing the LLM from overfitting to specific user data. This maintains the LLM's general language capabilities.
- **LoRA**: Finetuning the LLM with LoRA (Hu et al. 2022), along with the user encoder and projection layers, offers parameter efficiency preventing catastrophic forgetting and maintaining the LLM's core knowledge.
- **Proj**: Finetuning only the projection layers (LLM and user encoder frozen) assesses the minimal parameter tuning required for effective LLM contextualization.

3.4 Efficiency

Compared to text-prompt-based methods, USER-LLM offers substantial efficiency gains. First, it leverages pretrained weights of both the user encoder and LLM, improving model convergence speed with less trainable parameters. Additionally, USER-LLM condenses user activities into dense representations, requiring only a single token per event. This frees up the LLM's context window, leading to improved inference efficiency. Furthermore, USER-LLM leverages *cross-attention*, maintaining user embeddings at a smaller dimension than the LLM's model dimension, which significantly reduces computation during both training and inference.

To further optimize inference efficiency, USER-LLM integrates *Perceiver* (Jaegle et al. 2021) units into its projection layers. *Perceiver* is a Transformer-based architecture that utilizes a trainable latent query to extract relevant information from input embeddings through *cross-attention*. In USER-LLM, *Perceiver* effectively compresses the user embeddings into a compact format using a latent query with shorter sequence length. This compression reduces the number of tokens needed to represent the user history, further freeing up the LLM's context window. This, along with *Perceiver*'s ability to distill insights from noisy contexts, makes USER-LLM ideal for practical applications with extensive user histories.

4 Experiments

4.1 Datasets and Tasks

We evaluated our approach on three widely recognized datasets: MovieLens20M (Harper and Konstan 2015), Amazon Review (He and McAuley 2016), and Google Review (Li, Shang, and McAuley 2022; Yan et al. 2023). Notably, Amazon Review dataset is well known for its high sparsity and variability and contains a significantly smaller number of training and evaluation examples.

We generated training and test examples by applying a sliding window over each user's feature sequences (sorted by timestamps). This creates inputs containing N items, with the subsequent item as the label. To ensure model evaluation

Datasets	#Users	#Items	#Train	Seq Len
MovieLens20M	82,977	27,280	13,821,405	50
Google Review	80,080	270,721	3,387,375	50
Amazon Review	5,756	177,978	357,258	50

Table 1: Dataset statistics. #Test is equal to #Users.

consistency, the final example (representing the most recent interaction) is used as the test example for each user and is never included in the training set. Table 1 summarizes datasets statistics after processing.

To address the diverse evaluation needs, we accessed model performance on three distinct types of *open-text-generation* tasks. These tasks serve as a proof-of-concept, demonstrating USER-LLM's ability to understand user preferences from historical interactions and provide peronslized response accordingly.

- **Next item prediction** Given a historical sequence of items, the model predicts the subsequent item. For example, predicting the next movie a user watches based on a sequence of previously viewed movie IDs.
- **Favorite genre or category prediction** The model predicts a user's favorite genre or category based on a sequence of items (e.g., predicting favorite movie genre given a sequence of movie IDs). The favorite is the one with the highest frequency in the user's history.
- **Review generation** This task extends beyond simple predictions. Here, the model must generate actual reviews, utilizing multiple input features (e.g., movie ID, genre, and rating). These features are encoded to generate embeddings that guide the generation of a user's potential review for a given item.

4.2 Baselines and Experiment Setup

Our experiments used an Autoregressive Transformer for embedding generation. For co-training, we explored four different training strategies described in Section 3.3 and evaluated the impact of using a pretrained encoder versus a randomly initialized one.

We utilized PaLM-2 XXS (Anil et al. 2023) as the pretrained LLM. The user encoder is a 6-layer 128-dimensional transformer with 8 heads. We constructed the *Perceivers* module with 6 cross-attention layers and a query dimension of 16. We used $rank = 8$ for all LoRA tuning experiments.

To assess the effectiveness of USER-LLM, we compare it against the following two baselines:

- **Text Prompt (TP)** contextualize LLM by finetuning it with raw text input derived from a user's timeline.
- **Text Summarization (TS)** contextualize LLM by finetuning it with text Summarization derived from a user's timeline. In particular, we first use a Gemini-M model to generate summarizations for raw text inputs of user history, and then finetune the PaLM-2 XXS towards different tasks. This is a time consuming process, and due to resource limitation, we only conduct comparison on Amazon Review datasets.

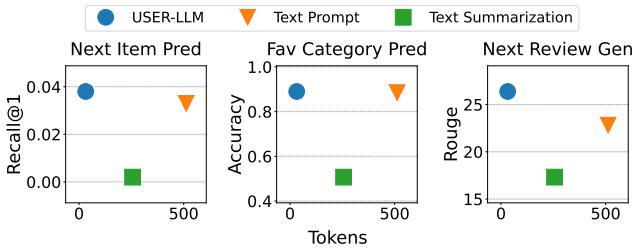

Figure 3: USER-LLM v.s. Text Prompt (TP) v.s. Text Summarization (TS) for 3 tasks on Amazon Review dataset. USER-LLM achieves better performance with much fewer tokens.

4.3 Performance and Efficiency Comparison

This subsection compares model accuracy and efficiency across various tasks and baselines. All reported USER-LLM experiments utilize pretrained encoders and finetune all three components (encoder, projection layers, and LLM) during cotraining, unless stated otherwise.

Overall Performance USER-LLM demonstrates significant efficiency gains (Table 2), achieving a remarkable **12-21.9X** reduction in FLOPs compared to the Text Prompt baseline. Moreover, USER-LLM consistently outperforms the Text Prompt baseline on most tasks across all three datasets, with performance improvements up to **16.33%**. The exception is the Google review next item task, where USER-LLM exhibits slightly lower performance.

While pretrained on next item prediction tasks, the user encoders generates embeddings encapsulating generalized user context, enabling USER-LLM to generalize well to diverse personalization tasks (e.g., favorite genre/categroy prediction, review generation) (Table 2). These tasks require deep user understanding, demonstrating USER-LLM's effectiveness in understanding user preferences from interaction data.

Fig. 3 compares the performance of USER-LLM against Text Prompt (TP) and Text Summarization (TS) baselines on three tasks using the Amazon Review dataset: Next Item Prediction, Favorite Category Prediction, and Next Review Generation. The x-axis represents the number of tokens used, highlighting the efficiency of each method. Notably, USER-LLM consistently achieves superior performance while utilizing significantly fewer tokens. This efficiency is evident in all three tasks, indicating the effectiveness of directly embedding user history as a distinct modality. USER-LLM requires fewer tokens to achieve equal or better performance, suggesting its ability to extract and leverage more relevant information from user data.

Varying Context Length To understand the impact of input length on performance and efficiency, we trained USER-LLM with varying predefined sequence lengths (50, 100, and 200 items) on MovieLens20M. Both frozen and tunable LLMs were evaluated and compared against a text-prompt baseline fine-tuned under the same computational constraints.

Table 4 highlights the significant computational advantage of USER-LLM. As the input sequence length increases, the required number of input tokens for the text-prompt approach

Dataset	Task	Metrics	Text-Prompt	User-LLM	Relative Improvement	FLOPs Reduction
MovieLens 20M	NextItem	Recall@5	0.140	0.154	10.00%	21.9X
	FavGenre	Accuracy	0.787	0.787	0.00%	21.9X
Google Review	NextItem	Recall@5	0.057	0.052	-8.77%	12X
	FavCategory	Accuracy	0.741	0.862	16.33%	12X
	ReviewGen	Rouge	10.20	11.72	14.90%	16X
Amazon Review	NextItem	Recall@5	0.042	0.047	11.90%	16X
	FavCategory	Accuracy	0.885	0.890	0.57%	16X
	ReviewGen	Rouge	22.82	26.38	15.59%	16X

Table 2: User-LLM vs. Text-Promp for a variety of tasks on efficiency and performance. Note that we use full finetune for User-LLM. Relative improvement is calculated as (User-LLM - TP) / TP * 100%.

scales proportionally (700, 1350, and 2500 tokens for lengths 50, 100, and 200 respectively). This leads to a substantial increase in computational cost and memory demands. In contrast, USER-LLM leverages a user encoder that compresses user activity sequences into compact embeddings. This allows the LLM to operate on a fixed-length input of only 32 tokens, regardless of the input sequence length. To illustrate the efficiency gain, we can approximate FLOPs using the heuristic $FLOPs \approx 6ND$ (Kaplan et al. 2020), where N represents the number of LLM parameters, and D represents the total number of training tokens ($D = BSL$, with batch size B, training steps S, and input length L). Given that the encoder and *Perceiver* parameters (N_e and N_p respectively) are significantly smaller than the LLM parameters ($N_e \ll N$ and $N_p \ll N$), their computational costs are considered negligible. Using this approximation, USER-LLM achieves a remarkable FLOPs reduction compared to the text-prompt baseline, reaching up to **78.1X** when the input sequence length reaches 200.

Frozen LLM Furthermore, USER-LLM excels at integrating LLMs while preserving their inherent knowledge. By utilizing a training strategy (Enc) that keeps the LLM frozen, we prevent catastrophic forgetting and maintain the LLM's pretrained capabilities. As demonstrated in Table 4, USER-LLM with a frozen LLM surpasses the performance of text-prompt-based methods utilizing both full fine-tuning and LoRA tuning. Impressively, it achieves comparable performance to USER-LLM trained with an unfrozen LLM. These findings highlight USER-LLM's ability to effectively contextualize user preferences while safeguarding the LLM's pretrained knowledge, effectively mitigating the drawbacks associated with text-prompt-based methods.

Parameter efficiency USER-LLM requires fewer tunable parameters to achieve competitive performance. Table 3 shows the model accuracy across multiple training strategies. Notably, the *Enc* approach (tuning encoder and projection layers and freezing the LLM) achieved comparable task accuracy to full finetuning while requiring significantly fewer tuned parameters. This is because it effectively leverages the pretrained weights of both the user encoder and LLM, minimizing the need for extensive parameter updates during training. Thus, our approach offers a parameter-efficient solution for personalizing LLMs, facilitating model tuning on resource-constrained devices.

Dataset	Task	Metric	Full	LoRA	Enc	Proj
MovieLens 20M	NextItem	Recall@1	**0.055**	0.051	0.050	0.023
	FavGenre	Recall@1	**0.787**	0.787	0.786	0.743
Google Review	NextItem	Recall@1	0.015	**0.016**	0.015	0.015
	FavCat	Recall@1	**0.862**	0.844	0.844	0.615
	ReviewG	Rouge	11.72	11.64	**11.76**	9.61
Amazon Review	NextItem	Recall@1	**0.037**	0.028	0.028	0.014
	FavCat	Recall@1	**0.890**	0.887	0.885	0.850
	ReviewG	Rouge	**26.38**	24.56	24.30	19.04

Table 3: Comparison between different training strategies.

		Len50	Len100	Len200
# Tokens	USER-LLM	32	32	32
	TP	700	1350	2500
FLOPs Reduction		21.9X	42.2X	**78.1X**
Performance improvement	Unfrozen LLM	1.10X	1.23X	**1.49X**
	Frozen LLM	2.78X	2.97X	**3.27X**

Table 4: LLM input token counts for USER-LLM v.s. TextPrompt(TP). FLOPs reduction and performance improvement achieved by USER-LLM. FLOPs reduction = FLOPs(TP) / FLOPs(USER-LLM). Performance improvement = Recall@1(USER-LLM) / Recall@1(TP).

4.4 Ablation Study

Benefit of pretraining We study the benefits of pretraining user encoders for downstream tasks. Results presented in columns 3 to 4 in Table 6 demonstrate that models utilizing pretrained encoders consistently outperform those with randomly initialized encoders across all tasks, suggesting that pretraining enables encoders to capture user preference effectively, providing valuable context to LLMs during cotraining.

Task	USER-LLM		+Perceiver	
	Full	Enc	Full	Enc
NextItem	0.055	0.050	0.054	0.054
FavGenre	0.787	0.786	0.784	0.784
Tokens	50		16 (**-68%**)	

Table 5: USER-LLM and its *Perceiver* variant effectively reduce the number of embedding tokens while maintaining competitive performance. Experiments are done on MovieLens 20M. Metric: Recall@1.

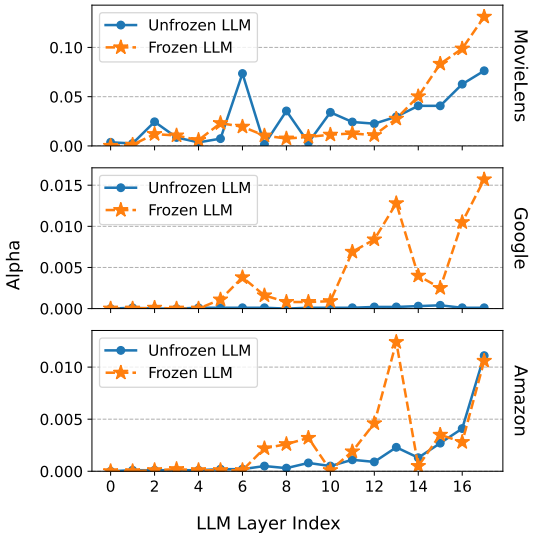

Figure 4: Converged gating parameters (Alpha) of different *cross-attention* layers indicate upper LLM transformer layers attend more signifcantly to encoder output than lower layers in next item prediction models on all three datasets.

Soft Prompt v.s. Cross Attention We compare the performance of USER-LLM with an alternative encoder-LLM integration strategy, *soft-prompt* (Lester, Al-Rfou, and Constant 2021). We set the task prompt's length to 10 and utilized a pretrained Autoregressive encoder in our experiment. As shown in the last column of Table 6, the cross-attention-based approach generally outperforms the soft-prompt approach. This advantage is particularly significant in the two review generation tasks, suggesting that the *cross-attention* mechanism enables LLMs to better extract and utilize the rich user information encoded within user embeddings. This highlights the value of cross-attention for tasks demanding a nuanced understanding of human intent.

Gated Cross-attention We further assess *cross-attention* in USER-LLM by employing gated cross-attention layers that include a trainable gating parameter α governing the amount of cross-attention to be added to the language model stack. Fig. 4 shows the converged values of gating parameters of 18 gated cross-attention layers for next item prediction tasks on MovieLens20M, Amazon Review and Google Local Review. The α values suggest that the upper LM layers attend more significantly to encoder output compared to the lower layers. Since the encoder output is user embedding that typically captures high-level semantics such as user attributes and interests, our result suggests that the upper layers in USER-LLM are likely to process such semantic information, hence attending more to encoder output. Interestingly, previous work by Geva and others (Geva et al. 2021) showed that lower layer keys in transformer correlate more with shallow surface features, whereas upper layer keys capture semantic topics. These analyses provide a plausible explanation why *cross-attention* outperforms *soft-prompt* in USER-LLM.

Dataset	Task	*cross-attention*		*soft prompt*
		PT	RD	
MovieLens 20M	NextItem	**0.055**	0.043	0.047
	FavGenre	**0.787**	0.782	**0.787**
Google Review	NextItem	**0.015**	**0.015**	0.013
	FavCat	**0.862**	0.853	0.822
	ReviewG	**11.72**	11.43	9.06
Amazon Review	NextItem	**0.037**	0.026	0.031
	FavCat	0.890	0.882	**0.894**
	ReviewG	**26.38**	23.12	25.80

Table 6: Experiments on different model fusing architectures, encoders, and the benefit of pretraining. Report Recall@1 for next item prediction and favorite genre/category prediction, ROUGE-Lsum for review generation. Full finetuning is used throughout the experiments. **PT**: Pretrained encoder. **RD**: Randomly initialized encoder. ***soft-prompt***: Connect user embeddings with LLM via soft-prompt.

5 Conclusion and Future Work

In this paper, we introduced USER-LLM, a framework for contextualizing LLMs through user embeddings. These embeddings, derived from self-supervised pretraining on diverse user interactions, capture hidden user preferences and their evolution. By integrating these embeddings with LLMs through *cross-attention*, USER-LLM empowers LLMs to adjust dynamically to user contexts.

Our comprehensive evaluation on MovieLens, Amazon Review, and Google Local Review datasets demonstrated that USER-LLM consistently outperforms text-prompt-based approaches on tasks requiring deep user understanding, particularly those involving long user interaction histories. USER-LLM's computational efficiency and ability to preserve LLM knowledge further make it particularly well-suited for practical applications where real-time personalization is crucial.

Future research directions include optimizing user embedding generation through advanced pretraining techniques and investigating the alignment between user embeddings and the language model for even deeper user context understanding. Additionally, training USER-LLM on a diverse range of tasks could further enhance its generalization abilities and adaptability across a broader spectrum of user scenarios. By addressing these aspects, USER-LLM has the potential to unlock the full power of LLMs for building truly personalized and adaptive AI agents.

References

Alayrac, J.-B.; Donahue, J.; Luc, P.; Miech, A.; Barr, I.; Hasson, Y.; Lenc, K.; Mensch, A.; Millican, K.; Reynolds, M.; et al. 2022. Flamingo: a visual language model for few-shot learning. *Advances in Neural Information Processing Systems*, 35: 23716–23736.

Anil, R.; Dai, A. M.; Firat, O.; Johnson, M.; Lepikhin, D.; Passos, A.; Shakeri, S.; Taropa, E.; Bailey, P.; Chen, Z.; et al. 2023. Palm 2 technical report. *arXiv preprint arXiv:2305.10403*.

Basyal, L.; and Sanghvi, M. 2023. Text Summarization Using Large Language Models: A Comparative Study of MPT-7b-instruct, Falcon-7b-instruct, and OpenAI Chat-GPT Models. arXiv:2310.10449.

Brown, T.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J. D.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; Agarwal, S.; Herbert-Voss, A.; Krueger, G.; Henighan, T.; Child, R.; Ramesh, A.; Ziegler, D.; Wu, J.; Winter, C.; Hesse, C.; Chen, M.; Sigler, E.; Litwin, M.; Gray, S.; Chess, B.; Clark, J.; Berner, C.; McCandlish, S.; Radford, A.; Sutskever, I.; and Amodei, D. 2020. Language Models are Few-Shot Learners. In Larochelle, H.; Ranzato, M.; Hadsell, R.; Balcan, M.; and Lin, H., eds., *Advances in Neural Information Processing Systems*, volume 33, 1877–1901. Curran Associates, Inc.

Cai, X.; Huang, C.; Xia, L.; and Ren, X. 2023. LightGCL: Simple Yet Effective Graph Contrastive Learning for Recommendation. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net.

Chen, S.; Wong, S.; Chen, L.; and Tian, Y. 2023a. Extending context window of large language models via positional interpolation. *arXiv preprint arXiv:2306.15595*.

Chen, X.; Wang, X.; Changpinyo, S.; Piergiovanni, A.; Padlewski, P.; Salz, D.; Goodman, S.; Grycner, A.; Mustafa, B.; Beyer, L.; et al. 2022a. Pali: A jointly-scaled multilingual language-image model. *arXiv preprint arXiv:2209.06794*.

Chen, Y.; Liu, Z.; Li, J.; McAuley, J. J.; and Xiong, C. 2022b. Intent Contrastive Learning for Sequential Recommendation. In Laforest, F.; Troncy, R.; Simperl, E.; Agarwal, D.; Gionis, A.; Herman, I.; and Médini, L., eds., *WWW '22: The ACM Web Conference 2022, Virtual Event, Lyon, France, April 25 - 29, 2022*, 2172–2182. ACM.

Chen, Y.; Qian, S.; Tang, H.; Lai, X.; Liu, Z.; Han, S.; and Jia, J. 2023b. Longlora: Efficient fine-tuning of long-context large language models. *arXiv preprint arXiv:2309.12307*.

Chevalier, A.; Wettig, A.; Ajith, A.; and Chen, D. 2023. Adapting Language Models to Compress Contexts. *arXiv preprint arXiv:2305.14788*.

Chidambaram, M.; Yang, Y.; Cer, D.; Yuan, S.; Sung, Y.-H.; Strope, B.; and Kurzweil, R. 2018. Learning cross-lingual sentence representations via a multi-task dual-encoder model. *arXiv preprint arXiv:1810.12836*.

Chowdhery, A.; Narang, S.; Devlin, J.; Bosma, M.; Mishra, G.; Roberts, A.; Barham, P.; Chung, H. W.; Sutton, C.; Gehrmann, S.; Schuh, P.; Shi, K.; Tsvyashchenko, S.; Maynez, J.; Rao, A.; Barnes, P.; Tay, Y.; Shazeer, N.; Prabhakaran, V.; Reif, E.; Du, N.; Hutchinson, B.; Pope, R.; Bradbury, J.; Austin, J.; Isard, M.; Gur-Ari, G.; Yin, P.; Duke, T.; Levskaya, A.; Ghemawat, S.; Dev, S.; Michalewski, H.; Garcia, X.; Misra, V.; Robinson, K.; Fedus, L.; Zhou, D.; Ippolito, D.; Luan, D.; Lim, H.; Zoph, B.; Spiridonov, A.; Sepassi, R.; Dohan, D.; Agrawal, S.; Omernick, M.; Dai, A. M.; Pillai, T. S.; Pellat, M.; Lewkowycz, A.; Moreira, E.; Child, R.; Polozov, O.; Lee, K.; Zhou, Z.; Wang, X.; Saeta, B.; Diaz, M.; Firat, O.; Catasta, M.; Wei, J.; Meier-Hellstern, K.; Eck, D.; Dean, J.; Petrov, S.; and Fiedel, N. 2023. PaLM: Scaling Language Modeling with Pathways. *Journal of Machine Learning Research*, 24(240): 1–113.

Covington, P.; Adams, J.; and Sargin, E. 2016a. Deep Neural Networks for YouTube Recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems*. New York, NY, USA.

Covington, P.; Adams, J.; and Sargin, E. 2016b. Deep Neural Networks for YouTube Recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems (RecSys'16)*, 191—-198.

Devlin, J.; Chang, M.-W.; Lee, K.; and Toutanova, K. N. 2018. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv preprint arXiv:1810.04805*.

Ding, J.; Ma, S.; Dong, L.; Zhang, X.; Huang, S.; Wang, W.; Zheng, N.; and Wei, F. 2023. Longnet: Scaling transformers to 1,000,000,000 tokens. *arXiv preprint arXiv:2307.02486*.

Doddapaneni, S.; Sayana, K.; Jash, A.; Sodhi, S.; and Kuzmin, D. 2024. User Embedding Model for Personalized Language Prompting. *arXiv preprint arXiv:2401.04858*.

Driess, D.; Xia, F.; Sajjadi, M. S.; Lynch, C.; Chowdhery, A.; Ichter, B.; Wahid, A.; Tompson, J.; Vuong, Q.; Yu, T.; et al. 2023. Palm-e: An embodied multimodal language model. *arXiv preprint arXiv:2303.03378*.

Ge, T.; Hu, J.; Wang, X.; Chen, S.-Q.; and Wei, F. 2023. In-context autoencoder for context compression in a large language model. *arXiv preprint arXiv:2307.06945*.

Gemini Team Google. 2023. Gemini: A family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*.

Geva, M.; Schuster, R.; Berant, J.; and Levy, O. 2021. Transformer Feed-Forward Layers Are Key-Value Memories. In Moens, M.-F.; Huang, X.; Specia, L.; and Yih, S. W.-t., eds., *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 5484–5495. Online and Punta Cana, Dominican Republic: Association for Computational Linguistics.

Gillick, D.; Kulkarni, S.; Lansing, L.; Presta, A.; Baldridge, J.; Ie, E.; and Garcia-Olano, D. 2019. Learning dense representations for entity retrieval. *arXiv preprint arXiv:1909.10506*.

Gillick, D.; Presta, A.; and Tomar, G. S. 2018. End-to-end retrieval in continuous space. *arXiv preprint arXiv:1811.08008*.

Girdhar, R.; El-Nouby, A.; Liu, Z.; Singh, M.; Alwala, K. V.; Joulin, A.; and Misra, I. 2023. Imagebind: One embedding space to bind them all. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 15180–15190.

Google, G. T. 2023. Gemini: A Family of Highly Capable Multimodal Models. arXiv:2312.11805.

Gu, A.; and Dao, T. 2023. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. arXiv:2312.00752.

Gu, A.; Johnson, I.; Timalsina, A.; Rudra, A.; and Ré, C. 2023. How to Train your HIPPO: State Space Models with Generalized Orthogonal Basis Projections. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net.

Han, J.; Gong, K.; Zhang, Y.; Wang, J.; Zhang, K.; Lin, D.; Qiao, Y.; Gao, P.; and Yue, X. 2023. Onellm: One framework to align all modalities with language. *arXiv preprint arXiv:2312.03700*.

Harper, F. M.; and Konstan, J. A. 2015. The MovieLens Datasets: History and Context. *ACM Trans. Interact. Intell. Syst.*, 5(4).

He, R.; and McAuley, J. 2016. Ups and Downs: Modeling the Visual Evolution of Fashion Trends with One-Class Collaborative Filtering. In *Proceedings of the 25th International Conference on World Wide Web*.

Hu, E. J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; and Chen, W. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*.

Jaegle, A.; Gimeno, F.; Brock, A.; Vinyals, O.; Zisserman, A.; and Carreira, J. 2021. Perceiver: General perception with iterative attention. In *International conference on machine learning*, 4651–4664. PMLR.

Ji, J.; Li, Z.; Xu, S.; Hua, W.; Ge, Y.; Tan, J.; and Zhang, Y. 2023. GenRec: Large Language Model for Generative Recommendation. arXiv:2307.00457.

Jiang, J.-Y.; Wu, T.; Roumpos, G.; Cheng, H.-T.; Yi, X.; Chi, E.; Ganapathy, H.; Jindal, N.; Cao, P.; and Wang, W. 2020. End-to-End Deep Attentive Personalized Item Retrieval for Online Content-sharing Platforms. In *Proceedings of The Web Conference 2020*, 2870–2877.

Jin, H.; Han, X.; Yang, J.; Jiang, Z.; Liu, Z.; Chang, C.-Y.; Chen, H.; and Hu, X. 2024. LLM Maybe LongLM: Self-Extend LLM Context Window Without Tuning. *arXiv preprint arXiv:2401.01325*.

Kang, W.-C.; Ni, J.; Mehta, N.; Sathiamoorthy, M.; Hong, L.; Chi, E.; and Cheng, D. Z. 2023. Do LLMs Understand User Preferences? Evaluating LLMs On User Rating Prediction. arXiv:2305.06474.

Kaplan, J.; McCandlish, S.; Henighan, T.; Brown, T. B.; Chess, B.; Child, R.; Gray, S.; Radford, A.; Wu, J.; and Amodei, D. 2020. Scaling Laws for Neural Language Models. arXiv:2001.08361.

Kojima, T.; Gu, S. S.; Reid, M.; Matsuo, Y.; and Iwasawa, Y. 2022. Large Language Models are Zero-Shot Reasoners. In Koyejo, S.; Mohamed, S.; Agarwal, A.; Belgrave, D.; Cho, K.; and Oh, A., eds., *Advances in Neural Information Processing Systems*, volume 35, 22199–22213. Curran Associates, Inc.

Lester, B.; Al-Rfou, R.; and Constant, N. 2021. The Power of Scale for Parameter-Efficient Prompt Tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*.

Li, J.; Shang, J.; and McAuley, J. 2022. UCTopic: Unsupervised Contrastive Learning for Phrase Representations and Topic Mining. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 6159–6169.

Li, R.; Deng, W.; Cheng, Y.; Yuan, Z.; Zhang, J.; and Yuan, F. 2023. Exploring the Upper Limits of Text-Based Collaborative Filtering Using Large Language Models: Discoveries and Insights. arXiv:2305.11700.

Liu, H.; Zaharia, M.; and Abbeel, P. 2023. Ring attention with blockwise transformers for near-infinite context. *arXiv preprint arXiv:2310.01889*.

Liu, J.; Liu, C.; Zhou, P.; Lv, R.; Zhou, K.; and Zhang, Y. 2023a. Is ChatGPT a Good Recommender? A Preliminary Study. arXiv:2304.10149.

Liu, Y.; Shi, K.; He, K. S.; Ye, L.; Fabbri, A. R.; Liu, P.; Radev, D.; and Cohan, A. 2023b. On Learning to Summarize with Large Language Models as References. arXiv:2305.14239.

Lyu, H.; Jiang, S.; Zeng, H.; Wang, Q.; Zhang, S.; Chen, R.; Leung, C.; Tang, J.; Xia, Y.; and Luo, J. 2023. LLM-Rec: Personalized Recommendation via Prompting Large Language Models. arXiv:2307.15780.

Ma, J.; Zhao, Z.; Yi, X.; Chen, J.; Hong, L.; and Chi, E. H. 2018. Modeling task relationships in multi-task learning with multi-gate mixture-of-experts. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 1930–1939.

Moon, S.; Madotto, A.; Lin, Z.; Nagarajan, T.; Smith, M.; Jain, S.; Yeh, C.-F.; Murugesan, P.; Heidari, P.; Liu, Y.; et al. 2023. Anymal: An efficient and scalable any-modality augmented language model. *arXiv preprint arXiv:2309.16058*.

Mu, J.; Li, X. L.; and Goodman, N. 2023. Learning to compress prompts with gist tokens. *arXiv preprint arXiv:2304.08467*.

Ning, L.; Singhal, K.; Zhou, E. X.; and Prakash, S. 2021. Learning Federated Representations and Recommendations with Limited Negatives. *ArXiv e-prints*.

OpenAI. 2023. GPT-4 Technical Report. *ArXiv*, abs/2303.08774.

Petrov, A. V.; and Macdonald, C. 2023. Generative Sequential Recommendation with GPTRec. arXiv:2306.11114.

Qiu, Z.; Wu, X.; Gao, J.; and Fan, W. 2021. U-BERT: Pre-training User Representations for Improved Recommendation. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35.

Radford, A.; Kim, J. W.; Hallacy, C.; Ramesh, A.; Goh, G.; Agarwal, S.; Sastry, G.; Askell, A.; Mishkin, P.; Clark, J.; et al. 2021. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, 8748–8763. PMLR.

Reed, S.; Zolna, K.; Parisotto, E.; Colmenarejo, S. G.; Novikov, A.; Barth-Maron, G.; Gimenez, M.; Sulsky, Y.;

Kay, J.; Springenberg, J. T.; et al. 2022. A generalist agent. *arXiv preprint arXiv:2205.06175*.

Sountsov, P.; and Sarawagi, S. 2016. Length bias in encoder decoder models and a case for global conditioning. *arXiv preprint arXiv:1606.03402*.

Sun, F.; Liu, J.; Wu, J.; Pei, C.; Lin, X.; Ou, W.; and Jiang, P. 2019. BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer. In Zhu, W.; Tao, D.; Cheng, X.; Cui, P.; Rundensteiner, E. A.; Carmel, D.; He, Q.; and Yu, J. X., eds., *Proceedings of the 28th ACM International Conference on Information and Knowledge Management, CIKM 2019, Beijing, China, November 3-7, 2019*, 1441–1450. ACM.

Tay, Y.; Dehghani, M.; Bahri, D.; and Metzler, D. 2022. Efficient Transformers: A Survey. *ACM Comput. Surv.*, 55(6).

Touvron, H.; Martin, L.; Stone, K.; Albert, P.; Almahairi, A.; Babaei, Y.; Bashlykov, N.; Batra, S.; Bhargava, P.; Bhosale, S.; Bikel, D.; Blecher, L.; Ferrer, C. C.; Chen, M.; Cucurull, G.; Esiobu, D.; Fernandes, J.; Fu, J.; Fu, W.; Fuller, B.; Gao, C.; Goswami, V.; Goyal, N.; Hartshorn, A.; Hosseini, S.; Hou, R.; Inan, H.; Kardas, M.; Kerkez, V.; Khabsa, M.; Kloumann, I.; Korenev, A.; Koura, P. S.; Lachaux, M.-A.; Lavril, T.; Lee, J.; Liskovich, D.; Lu, Y.; Mao, Y.; Martinet, X.; Mihaylov, T.; Mishra, P.; Molybog, I.; Nie, Y.; Poulton, A.; Reizenstein, J.; Rungta, R.; Saladi, K.; Schelten, A.; Silva, R.; Smith, E. M.; Subramanian, R.; Tan, X. E.; Tang, B.; Taylor, R.; Williams, A.; Kuan, J. X.; Xu, P.; Yan, Z.; Zarov, I.; Zhang, Y.; Fan, A.; Kambadur, M.; Narang, S.; Rodriguez, A.; Stojnic, R.; Edunov, S.; and Scialom, T. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288.

Tworkowski, S.; Staniszewski, K.; Pacek, M.; Wu, Y.; Michalewski, H.; and Miłoś, P. 2023. Focused transformer: Contrastive training for context scaling. *arXiv preprint arXiv:2307.03170*.

Volkovs, M.; Yu, G. W.; and Poutanen, T. 2017. DropoutNet: Addressing Cold Start in Recommender Systems. In *NIPS*, 4957–4966.

Wang, W.; Dong, L.; Cheng, H.; Liu, X.; Yan, X.; Gao, J.; and Wei, F. 2023. Augmenting Language Models with Long-Term Memory. *arXiv preprint arXiv:2306.07174*.

Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Ichter, B.; Xia, F.; Chi, E.; Le, Q.; and Zhou, D. 2023. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv:2201.11903.

Wu, L.; Zheng, Z.; Qiu, Z.; Wang, H.; Gu, H.; Shen, T.; Qin, C.; Zhu, C.; Zhu, H.; Liu, Q.; Xiong, H.; and Chen, E. 2023a. A Survey on Large Language Models for Recommendation. arXiv:2305.19860.

Wu, S.; Fei, H.; Qu, L.; Ji, W.; and Chua, T.-S. 2023b. Next-gpt: Any-to-any multimodal llm. *arXiv preprint arXiv:2309.05519*.

Xia, L.; Huang, C.; Huang, C.; Lin, K.; Yu, T.; and Kao, B. 2023. Automated Self-Supervised Learning for Recommendation. In Ding, Y.; Tang, J.; Sequeda, J. F.; Aroyo, L.; Castillo, C.; and Houben, G., eds., *Proceedings of the ACM Web Conference 2023, WWW 2023, Austin, TX, USA, 30 April 2023 - 4 May 2023*, 992–1002. ACM.

Xie, X.; Sun, F.; Liu, Z.; Wu, S.; Gao, J.; Zhang, J.; Ding, B.; and Cui, B. 2022. Contrastive learning for sequential recommendation. In *2022 IEEE 38th international conference on data engineering (ICDE)*, 1259–1273. IEEE.

Xu, P.; Ping, W.; Wu, X.; McAfee, L.; Zhu, C.; Liu, Z.; Subramanian, S.; Bakhturina, E.; Shoeybi, M.; and Catanzaro, B. 2023. Retrieval meets long context large language models. *arXiv preprint arXiv:2310.03025*.

Xu, S.; Hua, W.; and Zhang, Y. 2023. OpenP5: Benchmarking Foundation Models for Recommendation. *arXiv:2306.11134*.

Yan, A.; He, Z.; Li, J.; Zhang, T.; and McAuley, J. 2023. Personalized Showcases: Generating multi-modal explanations for recommendations. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2251–2255.

Yang, J.; Yi, X.; Zhiyuan Cheng, D.; Hong, L.; Li, Y.; Xiaoming Wang, S.; Xu, T.; and Chi, E. H. 2020. Mixed Negative Sampling for Learning Two-tower Neural Networks in Recommendations. In *Companion Proceedings of the Web Conference 2020*, 441–447.

Yang, Y.; Huang, C.; Xia, L.; Huang, C.; Luo, D.; and Lin, K. 2023. Debiased Contrastive Learning for Sequential Recommendation. In *Proceedings of the ACM Web Conference 2023*, WWW '23, 1063–1073. New York, NY, USA: Association for Computing Machinery. ISBN 9781450394161.

Yanga, Y.; Yuanc, S.; Cera, D.; Konga, S.-y.; Constanta, N.; Pilarc, P.; Gea, H.; Sunga, Y.-H.; Stropea, B.; and Kurzweila, R. 2018. Learning Semantic Textual Similarity from Conversations. *ACL 2018*, 164.

Yao, T.; Yi, X.; Cheng, D. Z.; Yu, F.; Chen, T.; Menon, A.; Hong, L.; Chi, E. H.; Tjoa, S.; Kang, J. J.; and Ettinger, E. 2021. Self-Supervised Learning for Large-Scale Item Recommendations. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*, CIKM '21, 4321–4330. New York, NY, USA: Association for Computing Machinery. ISBN 9781450384469.

Yi, X.; Yang, J.; Hong, L.; Cheng, D. Z.; Heldt, L.; Kumthekar, A. A.; Zhao, Z.; Wei, L.; and Chi, E., eds. 2019. *Sampling-Bias-Corrected Neural Modeling for Large Corpus Item Recommendations*.

Yu, J.; Wang, Z.; Vasudevan, V.; Yeung, L.; Seyedhosseini, M.; and Wu, Y. 2022. Coca: Contrastive captioners are image-text foundation models. arXiv 2022. *arXiv preprint arXiv:2205.01917*.

A More details about Datasets

- **MovieLens20M**[1] A widely used benchmark dataset (Harper and Konstan 2015) for evaluating personalization and recommendation algorithms. Interaction modalities are movie name, genre, and rating.
- **Google Local Review Dataset**[2] (Li, Shang, and McAuley 2022; Yan et al. 2023) This dataset contains review information on Google Maps up to Sep 2021. We used New York's data in our experiments. Interaction modalities are place name, place category, rating, and review.
- **Amazon Review Dataset**[3] (He and McAuley 2016) This is a series of product review datasets crawled from Amazon.com. We used the "Movie_and_TV" category for evaluation. Interaction modalities are product title, product category, rating, and review summary (a short opinion summary written by the user).

B Training Hyperparameters

In Table 7, we show training hyperparameters of USER-LLM, including learning rate, weight decay, batch size, and number of training steps. In addition, USER-LLM used the cosine learning rate decay across all experiments and used the first 10% of training steps for linear warmup.

C Encoder Architectures

USER-LLM uses a Transformer-based user encoder to generate user embeddings from ID-based feature sequences. To effectively fuse information from user interaction data, we explore and compare two fusion architectures: *early fusion* and *late fusion*. Following the example shown in Fig. 5, each item in the user interaction timeline has three features: name, rating, and category. Each feature has its own vocabulary. Items from each feature are mapped to an integer ID and have their own embedding representations. In an *early fusion* architecture, the i_{th} item's three features (name, rating, and category) are combined into a fused embedding f_i. The sequence of fused embeddings is then fed into the user encoder to generate user embeddings. On the other hand, a *late fusion* architecture employs three separate feature encoders to process each feature independently. The resulting embeddings are combined to a single user embedding at a later stage using fusion layers.

Autoregressive Transformer As described in Section 3.2, USER-LLM employs an Autoregressive Transformer as the *early fusion* encoder. Illustrated in Fig. 2, this model receives a sequence of user activities, where each activity is represented by multiple features (name, rating, and category). Individual features are first embedded, then concatenated and processed by a Transformer decoder. The Autoregressive Transformer Encoder generates a user embedding for each input item in the user's timeline.

Dual Encoder The late-fusion encoder we investigated is a dual encoder architecture with separate 'user' and 'label' towers. The 'user tower' utilizes Transformers and processes a series of input tokens, while the 'label tower' processes the subsequent token (label) within the sequence. Each encoder generates an embedding representation of its respective input, and the similarity between these embeddings is calculated for loss computation via a softmax layer. When integrating with LLMs, we utilize only the 'user tower' for user embedding generation. With the default setting, a Dual Encoder generates a single user embedding from an input sequence. The number of generated embeddings can be adjusted through the configuration of the fusion layers.

Autoregressive Transformers and Dual Encoders offer different advantages for user embedding generation. Autoregressive Transformers excel at capturing long-range dependencies and contextual relationships within sequential data. However, they produce a fixed number of output embeddings (equal to the input sequence length), potentially introducing inefficiencies in the later LLM integration stage, especially when the sequence length is long. Furthermore, training these models can be computationally intensive.

Compared to Autoregressive Transformers, Dual Encoders offer several advantages. The separate feature encoders within user towers allow independent processing of features, handling features with varying sequence lengths or misaligned timestamps and potentially enhancing information extraction. The configurable fusion layers provide flexibility in the number of generated embeddings. Additionally, Dual Encoders support diverse pretraining tasks beyond next item prediction, as user and label towers can process different features (e.g., name for the user tower and category for the label tower). However, Dual Encoders may be less effective at modeling the complex sequential patterns often present in user timelines. The optimal encoder choice depends on dataset characteristics, the desired level of contextual understanding, and a balance between computational cost and potential performance gain.

Table 9 compares the performance of using Autoregressive Transformer (AR) and Dual Encoder (DE) on three datasets and eight tasks, without computation constraint. AR consistently outperforms DE, across various training strategies and using both pretrained or randomly initialized encoders. Table 10 further analyzes the two encoders' performance when incorporating short-term history into the LLM prompt.

D Integration Approaches

As mentioned in 4.3, we explored the integration of user embeddings and LLMs in USER-LLM through two approaches: *cross-attention* and *soft-prompt*. In the cross-attention method, the output embeddings from the pretrained user encoder are cross-attended with the LLM's intermediate text representations, similar to how Flamingo (Alayrac et al. 2022) works. On the other hand, the *soft-prompt* approach leverages user embeddings from the pretrained encoder as a soft prompt for the LLM. This soft prompt is prepended to the regular text prompt embedding, providing additional context about the user's interests and preferences.

Table 11 presents a detailed comparison between *cross-attention* and *soft-prompt* approaches with various training strategies across all datasets and tasks.

[1] https://grouplens.org/datasets/movielens/

[2] https://datarepo.eng.ucsd.edu/mcauley_group/gdrive/googlelocal/

[3] https://nijianmo.github.io/amazon/index.html

Dataset	Task	Learning rate	Weight decay	Batch size	Training steps
MovieLens 20M	EncoderPretrain	1e-1	1e-1	32,768	10,000
	NextItem	1e-3	1e-3	8,192	10,000
	FavGenre	1e-3	1e-3	1,024	5,000
Google Review	EncoderPretrain	1e-4	1e-1	16,384	10,000
	NextItem	1e-3	1e-1	2,048	5,000
	FavCat	1e-3	1e-1	2,048	5,000
	ReviewG	1e-3	1e-1	1,024	5,000
Amazon Review	EncoderPretrain	1e-4	1e-1	12,800	7,000
	NextItem	1e-3	1e-1	8,192	10,000
	FavCat	1e-3	1e-1	1,024	5,000
	ReviewG	1e-3	1e-1	2,048	5,000

Table 7: Training hyperparameters.

Dataset	Recall	TextPrompt		USER-LLM	
		TP10	TP50	AR50	AR50TP10
MovieLens 20M	@1	0.049	0.049	0.055	**0.059**
	@5	0.142	0.140	0.154	**0.173**
	@10	0.211	0.206	0.243	**0.252**
Google Review	@1	**0.021**	**0.021**	0.015	**0.021**
	@5	0.060	**0.061**	0.052	**0.061**
	@10	0.082	**0.086**	0.071	0.082
Amazon Review	@1	0.038	0.034	0.037	**0.041**
	@5	0.047	0.042	0.047	**0.051**
	@10	0.050	0.047	0.051	**0.055**

Table 8: Experiments on short-term text prompt plus long-term embedding. **TP10**: TextPrompt baseline with 10 recent user events. **TP50**: TextPrompt baseline with 50 recent user events. **AR50**: With 50 user events as encoder inputs. **AR50TP10**: With 50 user events as encoder inputs and 10 most recent events in text format as LLM inputs.

E More Results

E.1 Autoregressive Encoder v.s. Dual Encoder

Table 10 compares Autoregressive Encoder and Dual Encoder on long-term and short-term user context tasks. Refer to Columns 4 and 12 in Table 9 for the performance of these two encoders when utilizing a pretrained encoder and using **Full** training strategy.

E.2 Training Strategies and Encoder Pretraining

Table 9 shows the performance of Autoregressive encoder based USER-LLM under different training strategies. It also compares results between using a pretrained user encoder and a randomly initialized user encoder in the encoder-LLM integration stage.

E.3 Cross-attention v.s. Soft-prompt

Table 11 compares the performance of two encoder-LLM integration strategies: *cross-attention* and *soft-prompt*.

E.4 Next Item Prediction

Table 12 show results comparing text-prompt-based approach and USER-LLM in next item prediction task on MovieLens20M dataset with varying input activity sequence length.

E.5 Long-term and short-term user context

USER-LLM can integrate both long-term and short-term user context. We can feed long-term user history into the user encoder to generate compact user embeddings while prompting LLM with short-term user interactions. As shown in Table 8, USER-LLM generally achieves better results when combining long-term and short-term user context for next item prediction tasks. Using 50 user activities for embedding generation and 10 most recent activities for prompting LLM, USER-LLM (*AR50TP10*) outperforms TextPrompt with only short-term context (*TP10*) by providing long-term user information and outperforms TextPrompt with long-term context (*TP50*) by extracting meaningful user representations efficiently. Compared to *AR50*, in which USER-LLM only uses long-term user data, short-term context provides the model (*AR50TP10*) with additional natural language information about item names. This leads to improved next item name prediction while preserving computational efficiency.

F Review Generation Examples

We presented some examples from Amazon/Google review generation tasks below. From these examples, we can observe personalized responses generated using USER-LLM exhibit greater alignment with user preferences and backgrounds compared to unpersonalized responses. While discrepancies remain between personalized responses using USER-LLM and the ground truth, USER-LLM demonstrates the ability to produce reviews that reflect specific user characteristics and inclinations more effectively.

- **Input prompt:** "Write a review for The Firm - Aerobic Body Shaping."
  - **Ground truth response:** "Save Your Money."
  - **Personalized response:** "Terrible Waste of Time."
  - **Unpersonalized response:** "Aerobic Workout."

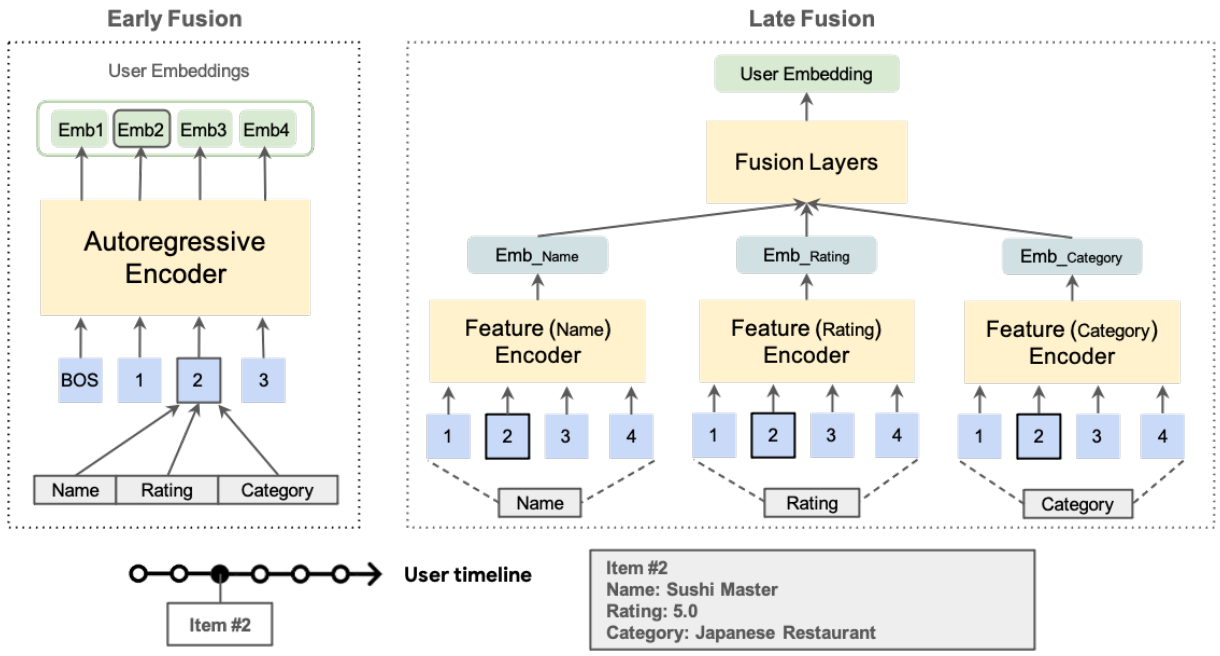

Figure 5: Encoder architectures supported in USER-LLM: *early fusion* and *late fusion* user encoders.

- **Input prompt:** "Write a review for A.I. - Artificial Intelligence."
  - **Ground truth response:** "A pure movie masterpiece of a film by Steven Speilberg. You will either love or hate this movie, but it will make you think."
  - **Personalized response:** "If you are into science fiction and you are looking for a movie that is not only good but also educational and fun, 'A.I."
  - **Unpersonalized response:** "A.I. - Artificial Intelligence."
- **Input prompt:** "Write a review for The Family that Preys."
  - **Ground truth response:** "Four Stars."
  - **Personalized response:** "Four Stars."
  - **Unpersonalized response:** "A great thriller!"
- **Input prompt:** "Write a review for Green Street Hooligans 2."
  - **Ground truth response:** "Hmmmmmm..."
  - **Personalized response:** "Not for everyone."
  - **Unpersonalized response:** "A good sequel."
- **Input prompt:** "Help me write a review for Blue with a rating of 5.0."
  - **Ground truth response:** "(Translated by Google) Very delicious and the service staff are very enthusiastic."
  - **Personalized response:** "(Translated by Google) Good food."
  - **Unpersonalized response:** "Great food and service."
- **Input prompt:** "Help me write a review for The Home Depot with a rating of 2.0."
  - **Ground truth response:** "(Translated by Google) The service is terrible (Original) El servicio esta pésimo."
  - **Personalized response:** "(Translated by Google) Very bad service (Original) Muy mal servicio."
  - **Unpersonalized response:** "No help."

	Dataset	Metric	AREnc-Xatten(USER-LLM)								DualEnc-Xatten					
			Full		LoRA		Enc		Proj		Full		Enc		Proj	
			PT	RD	PT	RD	PT	RD	PT	RD	PT	RD	PT	RD	PT	RD
Next Item	MovieLens 20M	R@1	**0.055**	0.041	0.051	0.043	0.050	0.041	0.023	0.002	0.044	0.025	0.034	0.010	0.032	0.001
		R@5	0.154	0.133	**0.164**	0.136	**0.164**	0.133	0.134	0.008	0.142	0.102	0.116	0.043	0.107	0.008
		R@10	0.243	0.203	**0.244**	0.205	0.243	0.204	0.203	0.013	0.218	0.169	0.182	0.076	0.166	0.012
	Google Review	R@1	0.015	0.015	**0.016**	0.015	0.015	0.015	0.015	0.015	0.014	0.015	0.014	0.015	0.015	0.015
		R@5	**0.052**	**0.052**	0.051	0.050	0.051	0.051	0.050	0.042	0.046	0.044	0.047	0.049	0.049	0.044
		R@10	**0.071**	**0.071**	0.070	0.070	0.070	0.070	0.070	0.057	0.066	0.063	0.061	0.067	0.064	0.057
	Amazon Review	R@1	**0.037**	0.023	0.028	0.018	0.028	0.022	0.014	0.000	0.020	0.005	0.016	0.002	0.008	0.000
		R@5	**0.047**	0.031	0.035	0.026	0.034	0.029	0.019	0.000	0.024	0.010	0.019	0.005	0.010	0.002
		R@10	**0.051**	0.037	0.038	0.029	0.036	0.030	0.021	0.0003	0.026	0.014	0.019	0.009	0.011	0.003
Fav Genre/Category	MovieLens 20M	R@1	**0.787**	0.778	**0.787**	0.779	0.786	0.778	0.743	0.484	0.775	0.704	0.776	0.759	0.649	0.484
		R@5	**0.996**	0.994	**0.996**	0.994	**0.996**	0.994	0.993	0.931	0.994	0.959	0.993	0.980	0.976	0.931
		R@10	0.999	0.999	0.999	0.999	0.999	0.999	0.999	0.991	0.999	0.987	0.999	0.997	0.998	0.991
	Google Review	R@1	**0.862**	0.853	0.844	0.785	0.844	0.775	0.615	0.142	0.797	0.642	0.760	0.508	0.271	0.142
		R@5	**0.951**	0.944	0.938	0.915	0.937	0.911	0.855	0.505	0.920	0.842	0.885	0.711	0.607	0.505
		R@10	**0.964**	0.956	0.948	0.941	0.949	0.936	0.909	0.660	0.944	0.871	0.914	0.798	0.763	0.661
	Amazon Review	R@1	**0.890**	0.882	0.887	0.884	0.885	0.886	0.850	0.223	0.854	0.745	0.843	0.714	0.312	0.223
		R@5	0.934	**0.948**	0.917	0.931	0.917	0.930	0.918	0.663	0.912	0.870	0.898	0.866	0.694	0.662
		R@10	0.944	**0.958**	0.919	0.934	0.919	0.932	0.923	0.793	0.934	0.925	0.930	0.903	0.846	0.793
Review Gen	Google Review	Rouge	11.72	11.43	11.64	11.55	**11.76**	11.56	9.61	8.05	11.71	11.04	**11.76**	10.82	9.18	6.62
	Amazon Review	Rouge	**26.38**	23.12	24.56	25.00	24.30	25.18	19.04	17.30	23.83	18.1	21.11	20.78	18.92	18.53

Table 9: USER-LLM experiments results with different training strategies and different user encoder architectures. **PT**: Pretrained user encoder. **RD**: Randomly initialized user encoder.

Dataset	**Recall**	**AREnc-Xatten**		**DualEnc-Xatten**	
		AR50	**AR50TP10**	**DE50**	**DE50TP10**
MovieLens 20M	@1	0.055	**0.059**	0.044	0.051
	@5	0.164	**0.173**	0.135	0.151
	@10	0.243	**0.252**	0.206	0.221
Google Review	@1	0.015	**0.021**	0.014	0.020
	@5	0.052	**0.061**	0.046	0.055
	@10	0.071	**0.082**	0.066	0.078
Amazon Review	@1	0.037	**0.041**	0.020	0.030
	@5	0.047	**0.051**	0.024	0.038
	@10	0.051	**0.055**	0.026	0.040

Table 10: Experiments on short-term text prompt long-term embedding with two encoder architectures: Autoregressive Encoder and Dual Encoder. **AR50**: With 50 user events as Autoregressive Encoder inputs. **AR50TP10**: With 50 user events as Autoregressive Encoder inputs and 10 most recent events in text format as LLM inputs. **DE50**: With 50 user events as Dual Encoder inputs. **DE50TP10**: With 50 user events as Dual Encoder inputs and 10 most recent events in text format as LLM inputs.

	Dataset	Metric	*cross-attention* (USER-LLM)				*soft-prompt*			
			Full	LoRA	Enc	Proj	Full	LoRA	Enc	Proj
NextItem	MovieLens20M	R@1	**0.055**	0.051	0.050	0.023	0.051	0.036	0.0047	0.0025
	Google Review	R@1	0.015	**0.016**	0.015	0.015	0.013	0.008	0.012	0.008
	Amazon Review	R@1	**0.037**	0.028	0.028	0.014	0.031	0.010	0.003	0.003
FavGenre (FavCat)	MovieLens20M	R@1	0.787	0.787	0.786	0.743	**0.787**	**0.787**	0.781	0.692
	Google Review	R@1	**0.862**	0.844	0.844	0.615	0.822	0.729	0.694	0.259
	Amazon Review	R@1	0.890	0.887	0.885	0.850	**0.894**	0.877	0.831	0.356
ReviewG	Google Review	ROUGE	11.72	11.64	**11.76**	9.61	9.06	9.04	8.75	7.76
	Amazon Review	ROUGE	**26.38**	24.56	24.30	19.04	25.80	22.86	20.36	15.12

Table 11: USER-LLM experiments results for *cross-attention* & *soft-prompt* with different training strategies (with pretrained Autoregressive encoders). Report Recall@1 for next item prediction and favorite genre/category prediction, ROUGE-Lsum for review generation.

Recall	**Unfrozen LLM**						**Frozen LLM**					
	Len50		**Len100**		**Len200**		**Len50**		**Len100**		**Len200**	
	TP	**USER-LLM**	**TP**	**USER-LLM**	**TP**	**USER-LLM**	**TP**	**USER-LLM**	**TP**	**USER-LLM**	**TP**	**USER-LLM**
@1	0.049	**0.055**	0.043	**0.055**	0.034	**0.055**	0.017	**0.051**	0.016	**0.051**	0.015	**0.053**
@5	0.140	**0.154**	0.123	**0.153**	0.102	**0.155**	0.051	**0.147**	0.046	**0.146**	0.042	**0.146**
@10	0.206	**0.227**	0.183	**0.226**	0.154	**0.229**	0.077	**0.214**	0.072	**0.214**	0.066	**0.216**

Table 12: Varying sequence length experiments on MovieLens20M with frozen LLM or unfrozen LLM. **Unfrozen LLM**: full finetune for TextPrompt(TP) model, training strategy ***Full*** for USER-LLM. **Frozen LLM**: LoRA parameters tuning for TextPrompt(TP) model, training strategy ***Enc*** for USER-LLM.

.